

Matchings

Design and Analysis of Algorithms

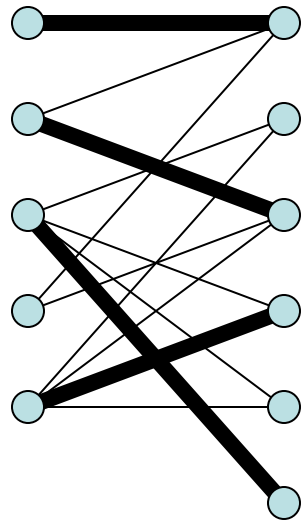
Brian C. Dean

**School of Computing
Clemson University**



Maximum Bipartite Matching

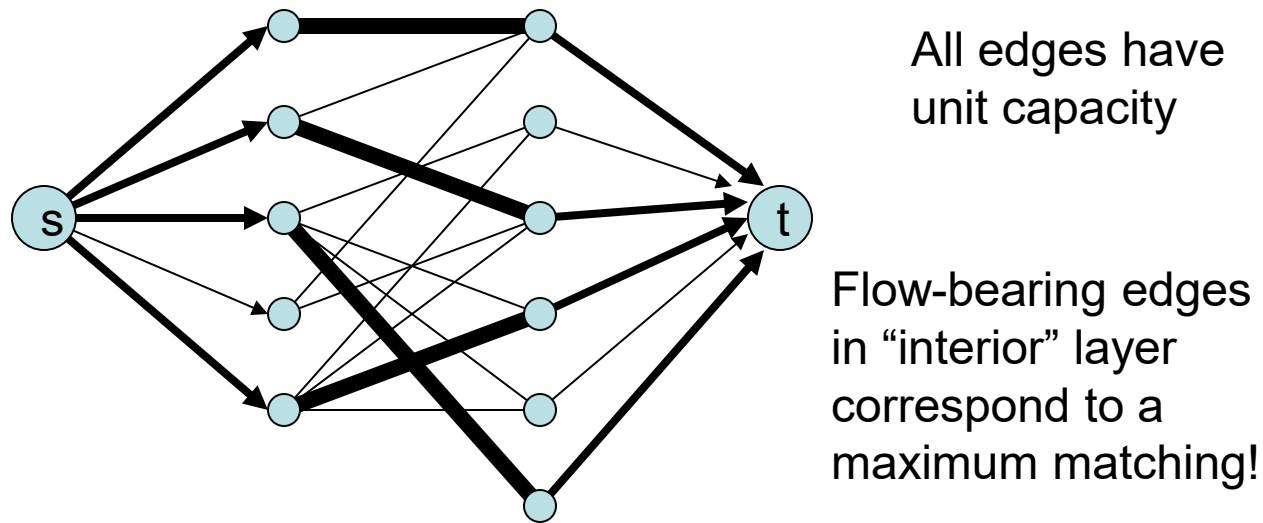
- Goal: find a matching containing the maximum possible number of edges.



- Is the matching above optimal?

Transformation into Max Flow

- Goal: find a matching containing the maximum possible number of edges.

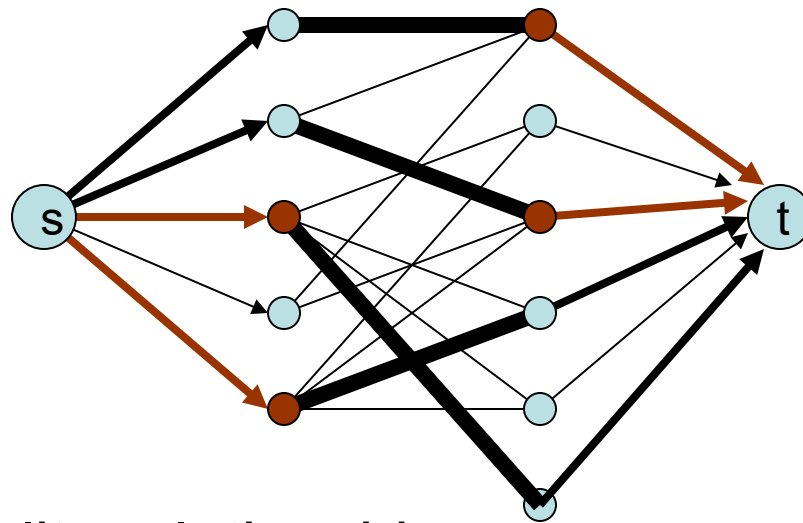


- $O(m)$ per augmentation times n augmenting paths:
 $O(mn)$ total time.

Duality with Vertex Cover

- Goal: find a matching containing the maximum possible number of edges.

The red edges cross a minimum $s - t$ cut



All edges have unit capacity

Flow-bearing edges in “interior” layer correspond to a maximum matching!

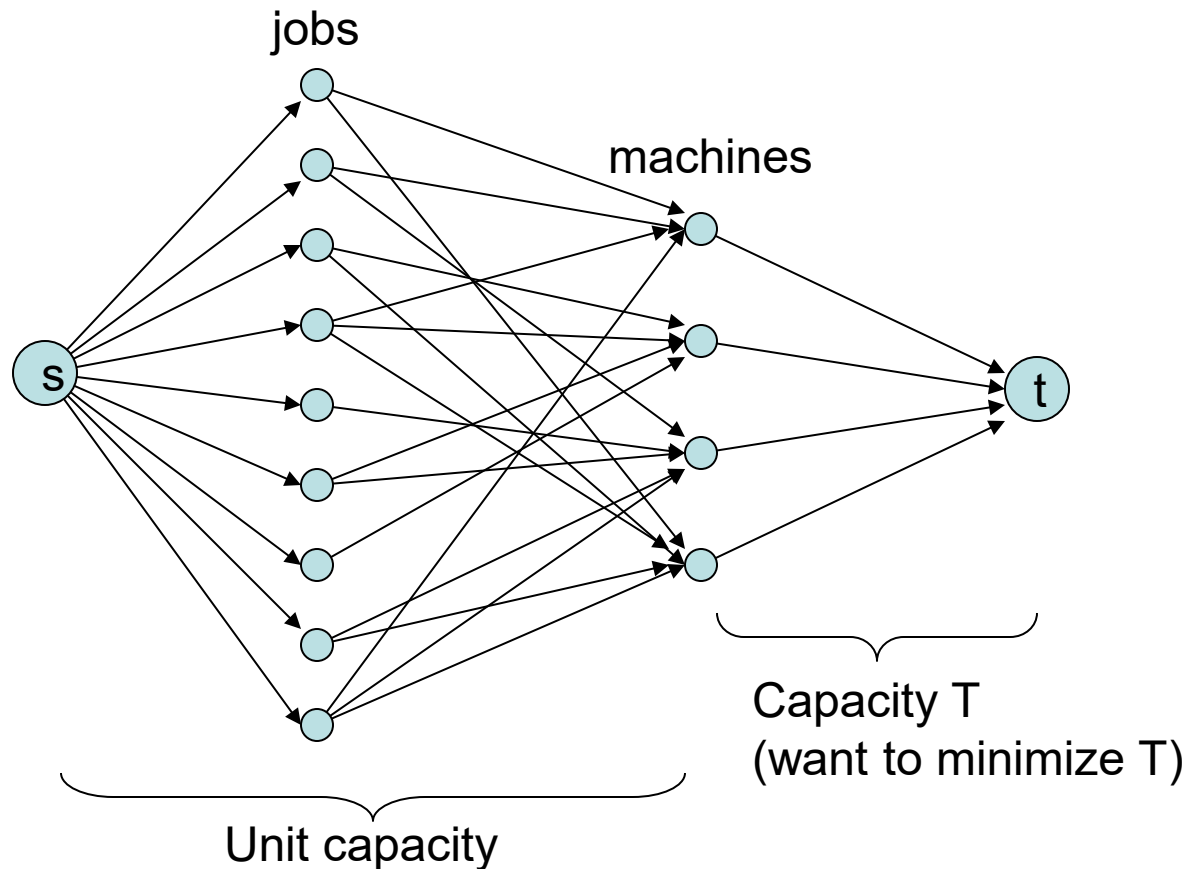
Another nice duality relationship:

In a general graph, $|\text{maximum matching}| \leq |\text{min vertex cover}|$

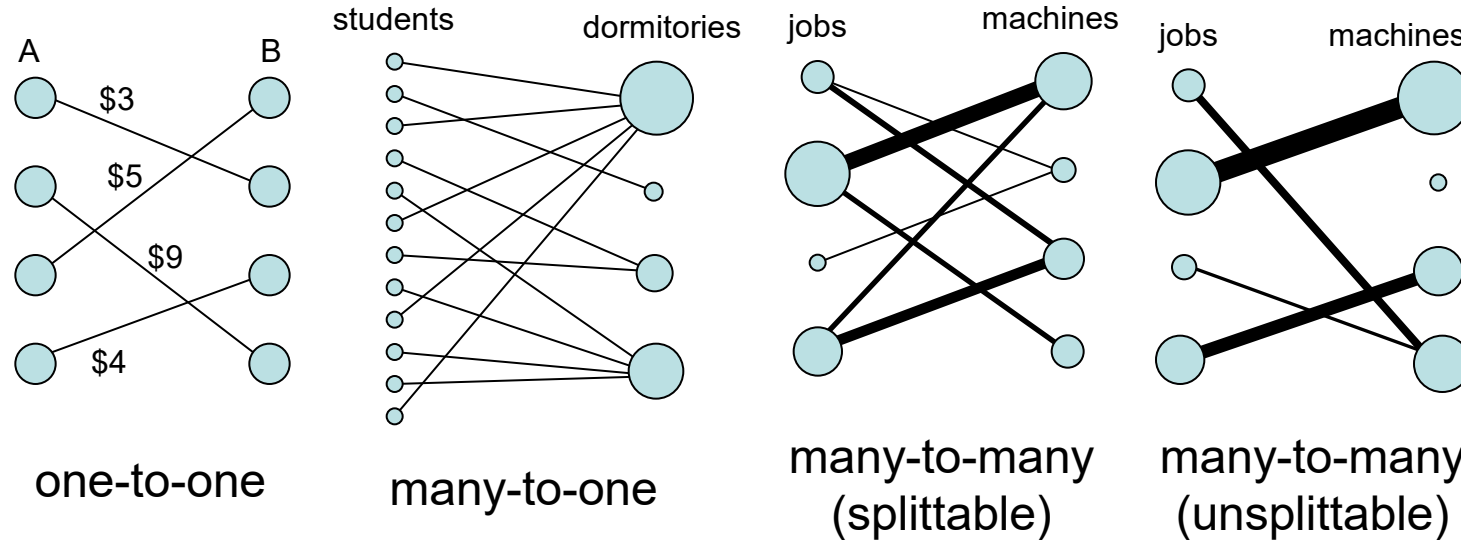
However, In a bipartite graph, $|\text{maximum matching}| = |\text{min vertex cover}|$
as a direct consequence of the maximum flow / minimum cut theorem!

Application: Load Balancing and Scheduling

- Each job takes 1 unit of time.
- **Goal:** finish all jobs as quickly as possible.



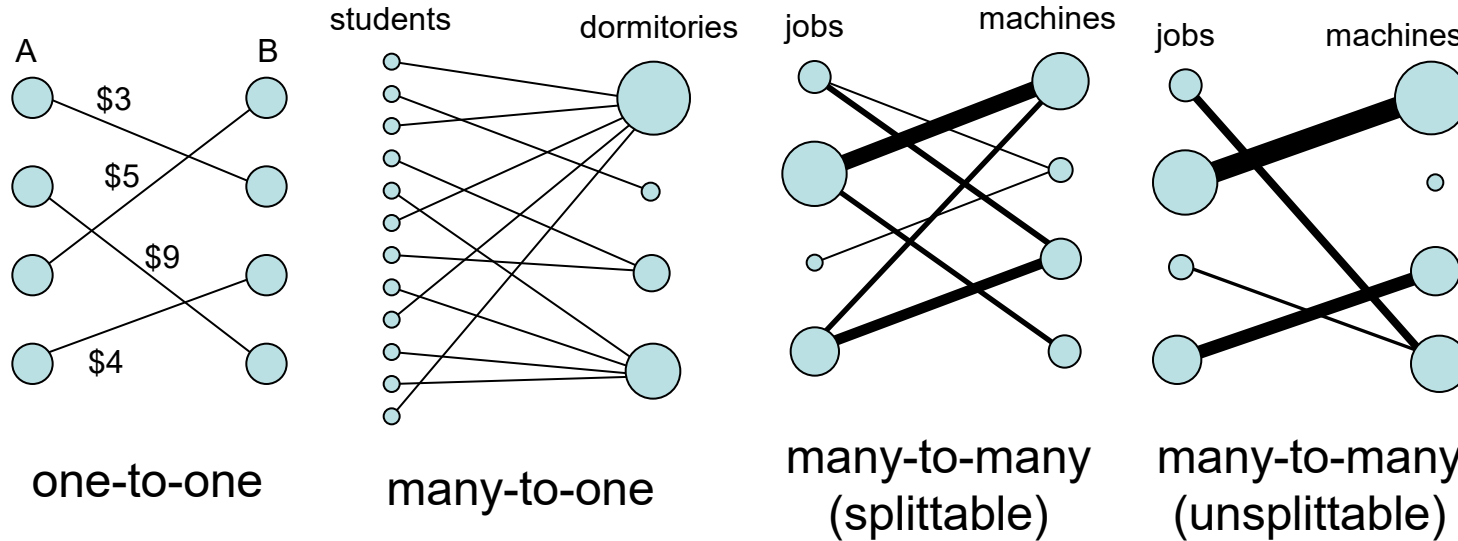
Bipartite Matching and Assignment Problems



Typical Objectives:

- **If no edge weights:** Compute a matching of maximum cardinality (i.e., match as much as possible)
- **If edges have weights:**
 - Compute an assignment of maximum value
 - Assign everything on the left-hand side at minimum cost.

Bipartite Matching and Assignment Problems

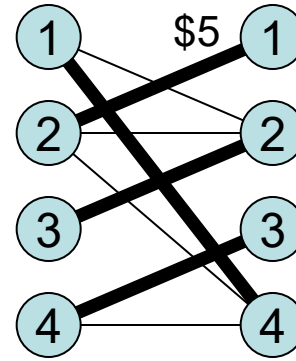


“Easy” to solve in polynomial time (e.g., using flow-based techniques)

NP-Hard!

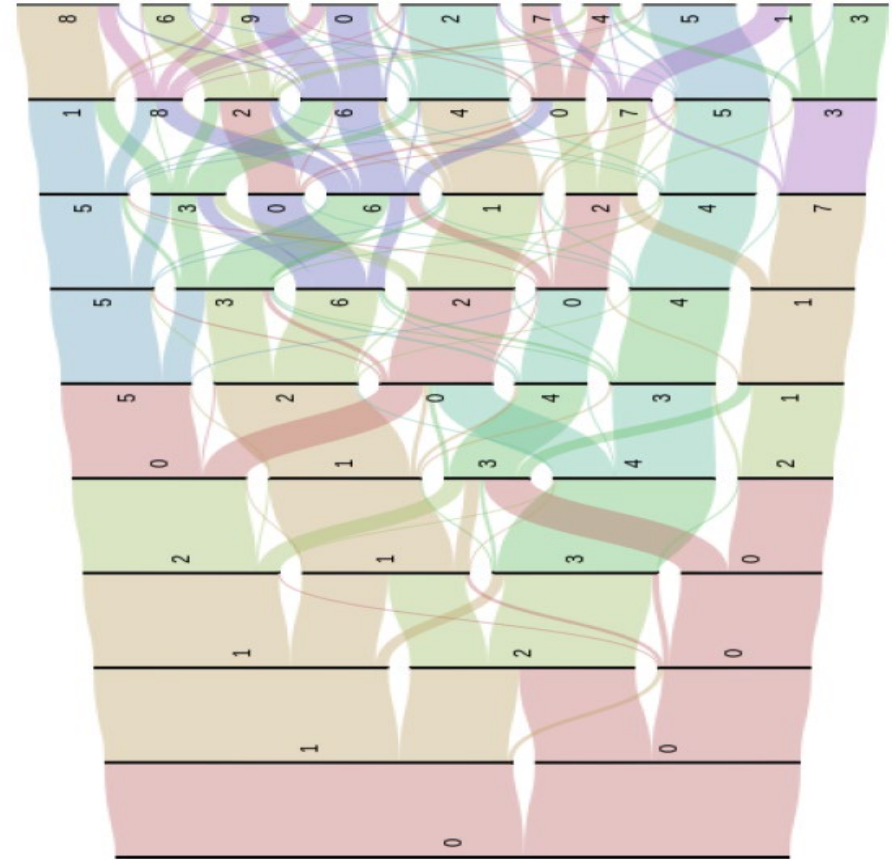
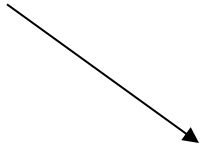
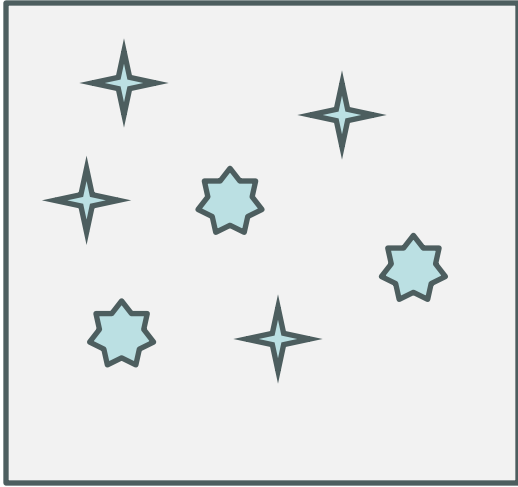
Min-Cost Perfect Matching (Bipartite Assignment)

0	7	0	1
5	9	0	8
0	2	0	0
0	0	3	6

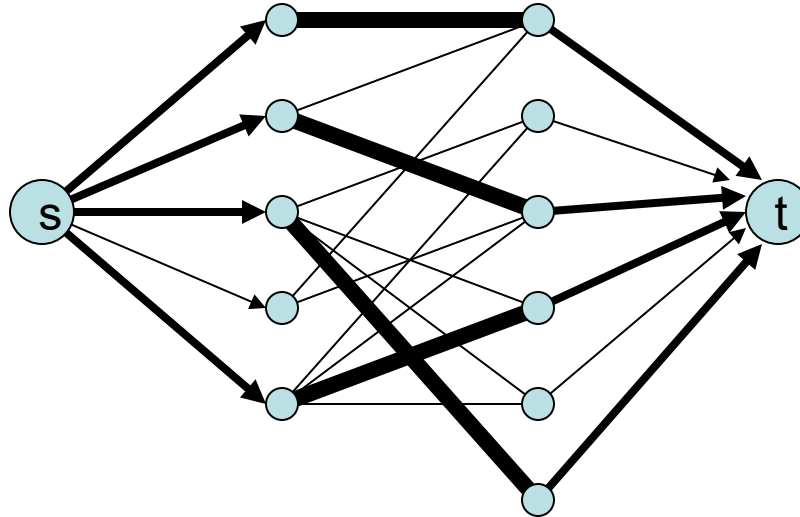


- Left and right side have same number of nodes.
- Given cost matrix C , where $C(i, j)$ is the cost of matching node i on the left with node j on the right.
- **Goal:** Match all nodes at minimum cost

Min-Cost Perfect Matching: Applications in Data Analytics



Min-Cost Perfect Matching: The $O(n^3)$ “Hungarian Algorithm”



- Use familiar transformation, giving a min-cost max flow problem.
- Optimally solved by augmenting on shortest (i.e., min-cost) $s \rightarrow t$ paths in our residual graph.
- Augmentation in $O(n^2)$ (Dijkstra in a dense graph), n augmentations.
- One subtlety: careful not to create negative-cost residual edges...